

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Índice general

5. Introducción a la Computación Paralela y Distribuida	1
5.1. Computación Paralela	1
5.2. Computación Distribuida	4
5.3. Grid Computing	5
5.4. Cloud Computing	7
5.5. Sistemas de Archivos Distribuidos	9
5.6. Tolerancia a fallos en Big Data	11
5.6.1. Tolerancia a fallos en batch	12
5.6.2. Tolerancia a fallos en Streaming	12
5.7. Hadoop	13
5.7.1. Principales Componentes	14
5.7.2. Ecosistema	16
5.8. Spark	18

Listado de acrónimos

DFS	Distributed File System
EC2	Elastic Compute Cloud
FLOPS	FLoating Point Operations Per Second
GFS	Google File System
HDFS	Hadoop Distributed File System
IAAS	Infrastructure as a Service
MIMD	Multiple Intruction Multiple Data
PAAS	Platform as a Service
SIMD	Simple Intruction Multiple Data
SAAS	Software as a Service
SOA	Service-oriented Architectures
VPC	Virtual Private Cloud
WORM	Write Once Read Many
YARN	Yet Another Resource Negotiator

5

Capítulo

Introducción a la Computación Paralela y Distribuida

5.1. Computación Paralela

Francisco P. Romero

Tanto el volumen de datos como la velocidad de procesamiento han seguido una trayectoria de aumento exponencial desde el inicio de la era digital, pero el primero ha aumentado a un ritmo mucho mayor que el segundo. De esta forma se hace imprescindible contar con nuevas herramientas como la computación o el procesamiento paralelo para conseguir salvar la brecha generada. Además de proporcionar una mayor capacidad de procesamiento para hacer frente a los requisitos de los grandes conjuntos de datos, el procesamiento paralelo tiene el potencial de aliviar el *cuello de botella de von Neumann* cuando los datos necesarios no pueden ser suministrados al procesador a la velocidad requerida.

Los algoritmos y arquitecturas de procesamiento paralelo se estudian desde desde la década de 1950 como una forma de mejorar el rendimiento de los sistemas de información y, más recientemente, como una forma de mejorar su rendimiento manteniendo el consumo de energía bajo control.

Los primeros supercomputadores seguían una arquitectura de flujo de instrucciones único, flujo de datos múltiple (SIMD) que utiliza una única unidad de ejecución de instrucciones, con cada instrucción aplicada a múltiples elementos de datos simultáneamente. La otra clase principal de arquitecturas paralelas se conoce como MIMD, en la que hay múltiples flujos de instrucciones además de múltiples flujos de datos. Esta última categoría cubre un amplio espectro de posibilidades dependiendo del modo de implementación de la memoria (global o distribuida) y cómo se comunican y sincronizan los múltiples procesadores entre sí (compartiendo memoria o mediante el paso de mensajes).

Los superordenadores modernos, incluida la mayoría de las entradas de la lista Top500 Supercomputers [KSV⁺21], tienden a ser MIMD lo que significa que utilizan memoria distribuida, por lo que los procesadores están dotados de memoria y se comunican mediante el paso de mensajes. Cada procesador tiene acceso directo (y por tanto rápido) a su memoria local y acceso indirecto (y por tanto más lento) a partes de memoria no locales o remotos. Los superordenadores de gama alta suelen utilizarse para realizar cálculos numéricos intensivos con números en coma flotante. Por ello, su rendimiento se mide en operaciones de coma flotante por segundo, o FLOPS.



Existen muchas taxonomías de máquinas paralelas con el fin de clasificar en grupos homogéneos las distintas arquitecturas que se van diseñando y produciendo en el tiempo. La más completa se debe a M. J. Flynn y la introdujo en 1966. Está basada en el número de flujos o canales para mover las instrucciones desde la memoria al procesador, único o múltiple y en el número de flujos existentes para trasladar los datos desde memoria al procesador, también único o múltiple.

La **ley de Moore** muestra el crecimiento exponencial en el número de transistores que pueden ser colocados en cada microchip. Hasta principios del siglo XXI, el aumento de la densidad de los chips iba acompañado de una mejora exponencial del rendimiento, debido a las correspondientes frecuencias de reloj más altas. Después, las líneas de tendencia de la frecuencia de reloj y el rendimiento comenzaron a aplanarse, sobre todo por el efecto de la disipación del calor, lo que dificultó la refrigeración de los circuitos superdensos. En ese momento, la atención a la mejora del rendimiento se trasladó al uso de múltiples procesadores independientes en el mismo chip, dando lugar a los procesadores multinúcleo. Utilizando c núcleos, cada uno con $\frac{1}{c}$, el rendimiento es más eficiente desde el punto de vista energético, dado que el consumo de energía es una función superlineal de el rendimiento de un solo núcleo.

La aparición del Big Data requiere una reevaluación de la forma en que medir el rendimiento de los superordenadores. Mientras que la clasificación de FLOPS la entrada/salida y el ancho de banda de almacenamiento asumen un papel más importante en la determinación del rendimiento. Un superordenador que realiza cálculos científicos puede recibir escasos parámetros de entrada y un conjunto de ecuaciones que definen un modelo y luego realizar cálculos durante días o semanas, antes de arrojar las respuestas. Mientras tanto, muchas aplicaciones Big Data requieren un flujo constante de nuevos datos que se introducen, se almacenan, se procesan y se ofrecen como resultados, con lo que posiblemente se ponga a prueba la memoria y el ancho de banda de E/S, capacidades que suelen ser más limitadas que la velocidad de cálculo.

En los últimos años, se han introducido muchos modelos abstractos de procesamiento paralelo en un esfuerzo por representar con precisión los recursos de procesamiento, almacenamiento y comunicación, junto con sus interacciones, permitiendo a los desarrolladores de aplicaciones paralelas visualizar y aprovechar las ventajas disponibles, sin tener que ocuparse en detalles específicos de la máquina. Con un alto nivel de abstracción podemos distinguir los enfoques de procesamiento paralelo de datos y de control.

El **paralelismo de datos** implica la partición de un gran conjunto de datos entre múltiples nodos de procesamiento, cada uno de ellos operando sobre una parte asignada de los datos, antes de que los resultados parciales se terminen combinando. A menudo, en las aplicaciones Big Data el mismo conjunto de operaciones debe ejecutarse en cada subconjunto de datos, por lo que el procesamiento SIMD es la alternativa más eficiente. En la práctica, sin embargo, la sincronización de un gran número de nodos de procesamiento introduce sobrecargas e ineficiencias que reducen las ganancias de velocidad. Por eso puede ser beneficioso dirigir los nodos de procesamiento para que ejecuten un único programa en múltiples de datos de forma asíncrona, con una coordinación dispersa.

En los esquemas basados en el **paralelismo de control** varios nodos operan independientemente resolviendo subproblemas, sincronizándose entre sí mediante el envío de mensajes informativos y de sincronización. Dicha independencia permite emplear recursos heterogéneos y recursos específicos de la aplicación sin interferencias cruzadas ni recursos más lentos obstaculicen el progreso de los más rápidos.



La **Ley de Amdahl** (Gene Amdahl, 1967) dice que a no ser que un programa secuencial pudiera ser completamente paralelizado, el *speedup* o aceleración que se obtendrá será muy limitado independientemente del número de núcleos disponibles.

🔗 Enlace: http://es.wikipedia.org/wiki/Ley_de_Amdahl



La **Ley de Gustafson** considera que la parte no paralela de un programa decrece cuando el tamaño del problema aumenta. Por lo tanto, cualquier problema suficientemente grande puede ser paralelizado eficientemente.

🔗 Enlace: http://es.wikipedia.org/wiki/Ley_de_Gustafson

5.2. Computación Distribuida

Un sistema distribuido es una colección de ordenadores autónomos enlazados por una red de ordenadores y soportados por un software que hace que la colección actúe como un servicio integrado [CDK05].

Algunos de los aspectos más importantes a tener en cuenta dentro de un sistema distribuido actual son los siguientes:

- **Apertura:** El sistema debe ser ampliable, es decir, debe existir la posibilidad de añadir nuevos recursos y servicios compartidos y que éstos se puedan poner a disposición de los diferentes componentes que forman el sistema.
- **Concurrencia:** Los distintos componentes de un sistema distribuido pueden demandar acceder a un recurso simultáneamente. Es necesario que el sistema esté diseñado para permitirlo.
- **Escalabilidad:** Un sistema es escalable si mantiene su eficiencia cuando hay un incremento significativo en el número de recursos y el número de usuarios.
- **Heterogeneidad:** El sistema distribuido puede estar formado por una variedad de diferentes redes, sistemas operativos, hardware, etc. A pesar de estas diferencias, los componentes deben poder interactuar entre sí.
- **Tolerancia a fallos:** Cualquier sistema puede fallar. En el caso de un sistema distribuido a pesar del fallo de un componente tiene que ser posible que el sistema siga funcionando.
- **Transparencia:** La transparencia permite que ciertos aspectos del sistema sean invisibles a las aplicaciones, por ejemplo, la ubicación, el acceso, la concurrencia, la gestión de los fallos, la replicación de la información, etc.

Los sistemas computacionales distribuidos son sistemas de computación de alto rendimiento que están formados por conjuntos de computadores interconectados mediante una red que ofrecen funcionalidades diversas algunas de ellas propias de la supercomputación, tales como la computación paralela. Entre estos sistemas cabe destacar los basados en *Grid* o en arquitecturas *Cloud* (en la nube) y de forma más básica, los denominados Clusters.

Los **Clusters** están formados por colecciones de computadores de similares características interconectados mediante una red de área local(ver Fig. 5.1). Los computadores hacen uso de un mismo sistema operativo y un middleware que se encarga de abstraer y virtualizar los diferentes computadores del sistema dando la visión al usuario de un sistema operativo único. Los *clusters* son sistemas dedicados a la supercomputación. El sistema operativo de un cluster es estándar y, por lo tanto, es el *middleware* quien provee de librerías que permiten la computación paralela.

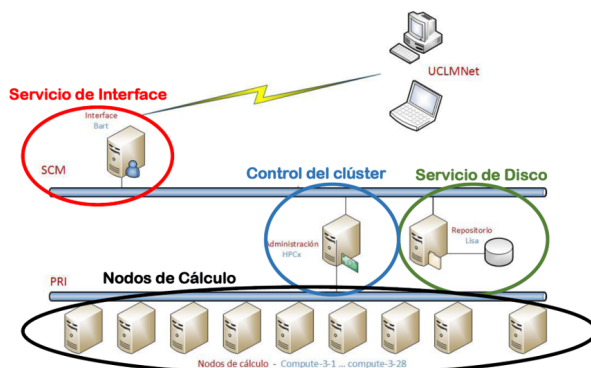


Figura 5.1: Esquema de organización de un servicio de supercomputación en Cluster . Fuente: Universidad de Castilla La Mancha

5.3. Grid Computing

El concepto de *Grid* (malla) proviene de los años 90 del siglo XX, cuando varios investigadores empezaron a plantear una infraestructura de computación que proporcionara acceso bajo demanda, seguro, flexible a los servicios de computación de altas prestaciones [Sin19].

El nombre de *Grid* se tomó por analogía con las redes eléctricas, con las que presentan algunos paralelismos, pero también importantes diferencias. Inicialmente el concepto de *Grid* se aplicó como paralelismo al hecho de que la electricidad en sí misma no supuso una revolución, sino más bien la creación de la red eléctrica que permitía un uso extensivo, barato y flexible de la electricidad. Otras propiedades, como la escalabilidad en función de la demanda son más propias de las redes eléctricas que de las redes de computación distribuida.

En [PF07] se define *Grid Computing* como una infraestructura persistente que soporta la computación y actividades de datos intensivas a través de múltiples organizaciones virtuales. Evidentemente, todas las implementaciones de Grid Computing se basan en la comunicación a través de Internet mediante distintos protocolos. Actualmente existen dos aproximaciones para la implementación de arquitecturas en Grid: Arquitecturas SOA y Arquitecturas Peer-to-Peer.

Las arquitecturas SOA (Fig. 5.2) están basadas en la agregación de servicios a los que se accede remotamente de forma independiente de las plataformas en las que se ejecutan y del lenguaje en los que están implementados. Un servicio es un programa autocontenido con una función y un interfaz bien definido. Este tipo de servicios se adapta muy bien a los sistemas *Grid* ya que ofrece, entre otras ventajas, una alta portabilidad al permitir interactuar servicios de sistemas heterogéneos a través de interfaces que son independientes de las plataformas donde están alojados; interoperabilidad a través de protocolos estándar de redes, y a su vez, permiten el uso de clientes ligeros que aíslan a los consumidores de la complejidad de los servicios.

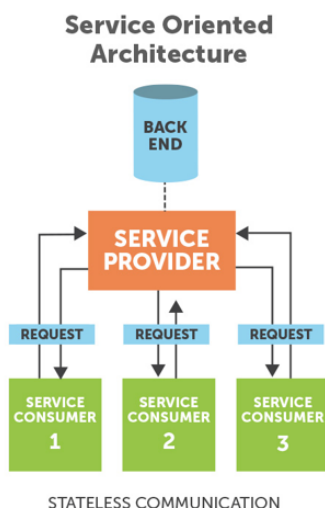


Figura 5.2: Esquema de una Arquitectura SOA

Las arquitecturas *Peer-To-Peer* son agregaciones de programas equivalentes que se ejecutan sobre plataformas heterogéneas y que comparten parte de su memoria y capacidad de cómputo. Estas plataformas presentan una alta tolerancia a errores, dado que cada nodo tiene la misma función que el resto. Este tipo de arquitecturas son más habituales en un tipo de computación conocido como *Volunteer Computing*.

Escalabilidad

La tecnología *Grid* permite de una forma muy fácil la escalabilidad, puesto que los nodos que actúan en ella son independientes a los demás. Esto permite que, si alguno de los nodos falla, se pueden delegar las tareas que estuviera realizando al resto de nodos, lo que permite seguir con la ejecución del programa y evitar paradas que puedan afectar a la efectividad de la obtención de los resultados deseados.

Además, este tipo de estructuración permite que se puedan escalar los recursos para cada computador o nodo de forma independiente en caso de que fuera necesario, sin tener que afectar al resto de computadoras. En el caso de actualizaciones que deban sufrir los equipos para obtener nuevos fragmentos de programas o mejoras acerca de ciertos aspectos, esta tecnología permite realizarlas de forma sencilla. Esto se consigue mediante la obtención de los recursos que vayan a verse afectados por estas actualizaciones y son puestas *offline* y retiradas de la red. Mientras tanto,

las tareas que suelen llevar son delegadas al resto de computadoras que se encuentren en las otras localizaciones. Esto permite que las actualizaciones se procedan en forma de cascada, haciendo que el trabajo en los proyectos que se estén llevando a cabo no se vea afectado.

Ventajas e Inconvenientes

Algunas de las ventajas que ofrece el *Grid Computing* son las siguientes:

- Es robusta en cuanto a sucesos que pudieran afectar a parte de su infraestructura, por ejemplo, catástrofes naturales. Los entornos *Grid* son bastante modulares e independientes, lo que reduce sus puntos vulnerables a fallo.
- Es una manera muy eficaz de utilizar los recursos de una manera óptima y eficiente en una organización.
- Puede utilizarse para balanceos de carga y conexiones de red que sean redundantes ofreciendo muchas facilidades en cuanto a la escalabilidad y actualización y siendo muy conveniente para ejecutar programas o tareas de forma paralela.

Algunas de las desventajas que tiene esta tecnología son: los riesgos de seguridad que ofrece la participación de diferentes entidades heterogéneas y la necesidad de una interconexión de altas prestaciones.

5.4. Cloud Computing

La computación en nube o **Cloud Computing** es una de las tecnologías actuales de mayor implantación. El concepto de computación en la nube supone el siguiente paso evolutivo de la computación distribuida superando los sistemas *legacy* y evolucionando hacia los sistemas distribuidos a gran escala. El objetivo de este modelo informático es hacer un mejor uso de los recursos distribuidos, ponerlos en común para lograr un mayor rendimiento y poder abordar problemas de computación a gran escala.



El acceso actual a la potencia de cálculo es extremadamente sencillo. Por ejemplo, el procesamiento de imágenes en Amazon Elastic Cloud Computing (EC2) [HK10] para el New York Times. La entrada de 11 millones de artículos (4 terabytes de datos TIFF) se procesa con éxito utilizando Amazon Simple Storage Service (Amazon S3) EC2 como hardware, y Hadoop con MapReduce como software. Los datos de salida son 1,5 TB en formato PDF, procesados en 24 horas con un coste de computación de tan solo 240 dólares.

La computación en la nube no es un concepto completamente nuevo para el desarrollo y funcionamiento de las aplicaciones web. Permite el desarrollo más rentable de portales web escalables en infraestructuras de alta disponibilidad y a prueba de fallos. En la computación en la nube se deben abordar diferentes retos como: la virtualización la escalabilidad, la interoperabilidad, la calidad del servicio, la tolerancia a errores y los modelos de nube.

Básicamente, la nube puede definirse mediante tres modelos distintos:

- **Nube privada:** Los datos y procesos se gestionan dentro de la organización sin las restricciones de ancho de banda de la red, los requerimientos de seguridad y los requisitos legales que supone el uso de los servicios de la nube pública a través de redes públicas y abiertas. Algunos ejemplos son Amazon VPC.
- **Nube pública:** Describe la computación en nube en el sentido tradicional, los recursos son facilitados de forma dinámica sobre una base de auto-servicio a través de aplicaciones web/servicios web, desde un proveedor externo que comparte recursos. Algunos ejemplos son Azure o Amazon EC2.
- **Nube híbrida:** El entorno está formado por múltiples proveedores internos y/o externos. Algunos ejemplos son RightScale, Asigra Hybrid Cloud Backup, Carpathia, Skytap y Elastra.

La arquitectura en la nube o *arquitectura cloud* subyacen en una infraestructura que se utiliza sólo cuando se necesita para extraer los recursos necesarios bajo demanda y realizar un trabajo específico, para luego ceder los recursos innecesarios. Los servicios son accesibles en cualquier parte del mundo, y la nube aparece como un único punto de acceso para todas las necesidades informáticas de los usuarios. Las arquitecturas en la nube abordan las principales dificultades que rodean el procesamiento de datos a gran escala. Existen diferentes categorías de servicios en la nube, como infraestructura, plataforma y aplicaciones. Estos servicios se prestan y consumen en tiempo real a través de Internet.

Software como servicio SAAS

El software como servicio es una plataforma multiusuario. Utiliza recursos comunes y una instancia única tanto del código objeto de una aplicación como de la base de datos subyacente para dar soporte a varios clientes simultáneamente. Este servicio es el nuevo método en la distribución de software de aplicaciones. Algunos ejemplos de los principales proveedores son Salesforce.com, NetSuite, Oracle, IBM y Microsoft, etc.

Plataforma como servicio PAAS

La plataforma como servicio proporciona a los desarrolladores una plataforma que incluye todos los sistemas y entornos, que comprende el ciclo de vida completo de desarrollo, prueba despliegue y alojamiento de aplicaciones web como un servicio prestado con una base en la nube. Proporciona una forma más fácil de desarrollar aplicaciones empresariales y diversos servicios a través de Internet. El

PAAS se plantea para facilitar el mantenimiento de la infraestructura de trabajo de los diferentes sistemas. Su aplicación debe reducir el tiempo de desarrollo al ofrecer un amplio abanico de herramientas y servicios fácilmente disponibles, y con una rápida capacidad de escalado.

Infraestructura como servicio IAAS

Infraestructura como servicio ofrece acceso basado en la web a almacenamiento y potencia de cálculo. Por ello, no se necesita gestionar o controlar la infraestructura subyacente de la nube, pero sí que se tiene control sobre los sistemas operativos, el almacenamiento y las aplicaciones desplegadas. Además de una mayor flexibilidad, una de las principales ventajas de IAAS es el esquema de pago basado en el uso. Esto permite a los clientes pagar a medida que crecen. Otra ventaja importante es la de utilizar siempre la última tecnología.

El camino hacia la computación en nube está impulsado por muchos factores, como la ubicuidad del acceso (todo lo que se necesita es un navegador), la facilidad de gestión (no hay necesidad de mejorar la experiencia del usuario, ya que no se necesita configuración o copia de seguridad), y una menor inversión (solución empresarial asequible desplegada sobre la base del pago por uso del hardware, con un software de sistemas proporcionado por los proveedores de la nube). Además, la computación en nube ofrece muchas ventajas a los proveedores, como una infraestructura fácil de gestionar dado que el centro de datos tiene un hardware y un software de sistema homogéneos.

5.5. Sistemas de Archivos Distribuidos

El sistema de archivos es un subsistema de un sistema operativo cuyo objetivo es organizar, recuperar y almacenar datos archivos. Un sistema de archivos distribuido (DFS) es un sistema de archivos con archivos compartidos en recursos de almacenamiento dispersos en una red. El DFS hace fácil la compartición de archivos entre aplicaciones cliente de forma controlada y autorizada. Asimismo, los usuarios pueden beneficiarse de los servicios de un DFS ya que pueden localizar todos los archivos compartidos dentro de un único servidor o nombre de dominio.

Una gran variedad de aplicaciones de análisis Big Data dependen de entornos distribuidos para analizar grandes cantidades de datos. A medida que aumenta la cantidad de datos, la necesidad de proporcionar soluciones de almacenamiento fiables y eficientes se ha convertido en una de las principales preocupaciones de los administradores de infraestructuras de Big Data. Los sistemas y métodos tradicionales de almacenamiento no son óptimos debido a sus restricciones de precio o rendimiento, mientras que el DFS supone se ha desarrollado con el fin de facilitar la compartición de archivos tanto en redes locales (LAN) como en redes de área amplia (WAN).

Una de las principales características de los DFS es la **transparencia** que hace que los archivos se lean, se almacenen y se gestionen en las máquinas cliente, mientras que el procesamiento real se produce en los servidores, es decir, el DFS implementa sus políticas de control de acceso y almacenamiento a sus clientes de forma centralizada proporcionando su servicios a través de la red.

El objetivo principal de la transparencia en el DFS es ocultar a los clientes el hecho de que los procesos y los recursos están distribuidos físicamente en la red y proporcionar una visión común de un sistema de archivos centralizado. Es decir, el DFS pretende ser *invisible* para los clientes, que consideran que el DFS es similar a un sistema de archivos local.

Además de la transparencia, otro de los factores fundamentales en los DFS es la **fiabilidad**. Dado que el lugar de uso de los archivos puede ser diferente de su lugar de almacenamiento, los modos de fallo son sustancialmente más complejos en los DFS en comparación con los sistemas de archivos locales. La replicación y la codificación de borrado son dos técnicas típicas para lograr una alta fiabilidad.

Los DFS utilizan la replicación haciendo varias copias de un archivo de datos en diferentes servidores. Cuando un cliente solicita el archivo, accede de forma transparente a una de de las copias. Dentro de la estrategia de **replicación** la ubicación de las réplicas juega un papel crítico en tanto en el rendimiento como en la fiabilidad de los DFS. Las réplicas se almacenan en diferentes servidores según de acuerdo con un esquema de colocación. Muchos DFS (por ejemplo HDFS [SKRC10]) utiliza por defecto la replicación aleatoria en la que las réplicas se almacenan aleatoriamente en diferentes nodos, en diferentes *racks*, en diferentes ubicación geográfica, de modo que si se produce un fallo en cualquier parte del sistema, los datos siguen estando disponibles.

Sin embargo, utilizando esta estrategia, no se puede separar eficazmente el *clúster* en niveles mientras se mantiene la consistencia del mismo y, por lo tanto, es bastante susceptible de a la pérdida frecuente de datos debido a fallos. La implementación de Facebook HDFS [BGS⁺11] limita la colocación de réplicas de bloques a grupos más pequeños de nodos, lo que reduce la probabilidad de pérdida de bloques con múltiples fallos de nodos.

El procedimiento de replicación dispersa múltiples copias de un archivo, y los cambios tienen que ser propagarse a todas las réplicas. Esto hace que la **consistencia de los datos** sea un aspecto clave en el funcionamiento de los DFS. El primer método para asegurar la consistencia de la caché es *escribir una vez leer muchos* (WORM - write once read many) en la que los archivos en caché están en modo de sólo lectura (es decir, una vez que se crea un archivo creado, no puede ser modificado) de manera que sólo la última versión de los datos puede ser leída. El segundo método es el *bloqueo transaccional* (*transactional locking*), en el que se obtiene un bloqueo de lectura en el archivo de datos solicitado, de forma que cualquier otro cliente no puede escribir este archivo o se obtiene un bloqueo de escritura para impedir cualquier acceso a este archivo, permitiendo que cada lectura refleje la última

escritura y que cada escritura se realice en orden. El tercer método es el *leasing* en el que el cliente tiene la garantía de que ningún otro cliente puede modificar los datos cuando se solicita el archivo de datos durante el *leasing*. Cuando éste expira o el cliente libera sus derechos, el archivo vuelve a estar disponible.

HDFS (Hadoop File System) es uno de los sistemas de archivos distribuidos más populares en la actualidad. Sus características principales son las siguientes:



- **Arquitectura:** Un clúster HDFS consiste en un único nodo de nombres (*namenode*, que es el nodo maestro que gestiona el espacio de nombres del sistema de archivos y regula el acceso a los archivos por parte de los clientes. El flujo de datos se divide en bloques distribuidos entre los *datanodes*, que gestionan el almacenamiento en los nodos en los que se ejecutan.
- **Acceso:** HDFS utiliza una librería de código que permite a los clientes leer, escribir y eliminar archivos y crear y eliminar directorios.
- **Replicación:** HDFS divide los datos en bloques que se replican a través de los *datanodes* de acuerdo con una política de colocación, por la cual cada *datanode* tiene como máximo una copia de un bloque y cada rack tiene como máximo dos copias del bloque.
- **Tolerancia a fallos:** Cada *datanode* compara la versión del software y el ID del espacio de nombres con los del *namenode*. Si no coinciden, el *datanode* se apaga para preservar la integridad del sistema.

5.6. Tolerancia a fallos en Big Data

Todos los sistemas de Big Data necesitan manejar los fallos de software y hardware que se producen en el sistema después de su desarrollo, lo que beneficiará a los sistemas de diferentes maneras, incluyendo: recuperación de fallos, menor coste, mayor rendimiento, etc. La tolerancia a fallos es una configuración que evita que un ordenador o dispositivo de red falle en caso de un problema o error inesperado[JPS12]. Para hacer que un computador o una red tolerante a fallos requiere que los usuarios o las empresas piensen en cómo puede fallar un ordenador o dispositivo de red y tomen medidas que ayuden a prevenir ese tipo de fallos[DK14].

El comportamiento de los sistemas Big Data se puede dividir en:

- **Batch Processing:** Procesamiento por lotes de datos en reposo. En este escenario, los datos de origen se cargan en los dispositivos de almacenamiento de datos, ya sea por la propia aplicación de origen o por un flujo de trabajo de orquestación (*orchestration workflow*). A continuación, los datos se procesan en el lugar por un trabajo paralelizado, que también puede ser iniciado por el flujo de trabajo de orquestación.

- *Stream Processing*: Procesamiento continuo de datos en tiempo real, es decir, en cuando existe disponibilidad de datos estos se procesan de manera secuencial. Se establecen unos flujos de datos infinitos y sin límites de tiempo.

Dentro de estos dos escenarios se pueden definir diferentes estrategias de tolerancia a fallos como se explica a continuación.

5.6.1. Tolerancia a fallos en batch

En el modelo de cálculo por lotes de *Hadoop*, las aplicaciones de dos principales mecanismos para hacer frente a los fallos son la replicación de datos y el mecanismo de rollback (reversión) [WDDL16].

- **Replicación de datos (Data Replication)**: En el mecanismo de replicación de datos una copia de los datos estará en varios nodos de datos diferentes. Cuando se necesita la replicación de datos, cualquier nodo de datos, cuya comunicación no esté ocupada puede copiar los datos. La principal ventaja de esta tecnología es que puede recuperarse instantáneamente de un fallo. El inconveniente es que se produce un consumo de una alta cantidad de recursos y la posibilidad de que los datos sean inconsistentes.
- **Mecanismo de reversión (Rollback Mechanism)**: El informe de la copia (*copy report*) se guardará periódicamente. Si se produce un fallo el sistema se limita a volver a un punto de control (*checkpoint*) y, a continuación, vuelve a iniciar la operación desde ese punto. El método adopta el concepto de *rollback*, es decir, el sistema volverá al trabajo anterior. Este método aumenta el tiempo de ejecución de todo el sistema, porque el *rollback* necesita hacer una copia de seguridad y comprobar que se guarda un estado consistente. En comparación la replicación supone más tiempo pero menos recursos.

5.6.2. Tolerancia a fallos en Streaming

En el sistema de *streaming*, hay cuatro tipos de estrategia para realizar la gestión de los fallos: la espera pasiva, la espera activa, la copia de seguridad ascendente.

- **Espera pasiva (Passive Standby)**: El sistema hará regularmente una copia de seguridad del último estado en el nodo maestro a una copia del nodo réplica. Cuando se produzca un fallo, el estado del sistema se restaurará a partir de los datos de la copia de seguridad. La estrategia de replicación pasiva admite el caso de que la carga de datos sea mayor, pero el tiempo de recuperación se incrementa. Los datos de copia de seguridad pueden guardarse en un sistema de almacenamiento distribuido para reducir el tiempo de recuperación.

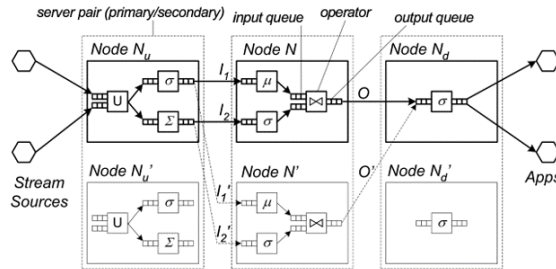


Figura 5.3: Esquema de nodos en streaming. Upstream N_u y downstream N_d . Fuente: [WDDL16]

- **Espera activa (Active Standby):** Cuando el sistema transmite datos para el nodo maestro, también transmite una copia de los datos para una réplica del nodo al mismo tiempo. Cuando el nodo maestro falla, una de las réplicas del nodo asume completamente el trabajo, y los nodos suplentes necesitan la asignación de los mismos recursos del sistema. De esta manera, el tiempo de recuperación del fallo es más corto, pero el rendimiento de los datos es menor. También desperdicia más recursos del sistema.
- **Copia de seguridad ascendente (Upstream Backup).** Cada nodo maestro almacena su propio estado y los datos de salida en un archivo de registro. Cuando un nodo maestro falla, el nodo maestro anterior en el flujo (*upstream*) reproducirá una copia de los datos en un archivo de registro al nodo correspondiente con el fin de recalculas las operaciones. Esta estrategia necesita más tiempo para reconstruir el estado de la recuperación, por lo que el tiempo de recuperación de fallos tiende a ser largo. La estrategia de copia de seguridad ascendente es una mejor opción en unas circunstancias de escasez de recursos.

5.7. Hadoop

Apache Hadoop es un framework de software que soporta aplicaciones distribuidas bajo una licencia libre. Permite a las aplicaciones trabajar con miles de nodos y petabytes de datos. Fue inicialmente concebido para resolver un problema de escalabilidad en *Nutch*, un motor de búsqueda Open Source. Al mismo tiempo, Google había publicado los documentos que describían su novedoso sistema de archivos distribuidos, el Google File System GFS), y *MapReduce*, un framework de computación para procesamiento paralelo. La exitosa implementación de estos conceptos en *Nutch* resultó en dos proyectos separados, el segundo se convirtió en *Hadoop*, un proyecto Apache de primera clase [Whi12].

El nombre de *Hadoop* no es ningún acrónimo, sino un nombre inventado. Su creador se lo puso por un elefante amarillo de peluche que tenía su hijo. Pensó que un nombre corto, relativamente fácil de deletrear y pronunciar sería adecuado.

Hadoop propone una arquitectura distribuida maestro/esclavo que está formada por los siguientes componentes: HDFS para almacenamiento de datos; YARN que se introdujo en Hadoop 2, que es un planificador de propósito general y administrador de recursos. Una aplicación YARN puede ejecutarse en un clúster Hadoop. *MapReduce*, un motor computacional batch implementado como una aplicación YARN.

Las características intrínsecas de Hadoop son la partición de datos y la posibilidad de cálculo paralelo sobre grandes conjuntos de datos. Sus capacidades de almacenamiento y computo se pueden ir ampliando con la incorporación de nuevos *hosts* a un clúster Hadoop. De esta forma, los clústeres con cientos de hosts pueden alcanzar fácilmente volúmenes de datos muy grandes.

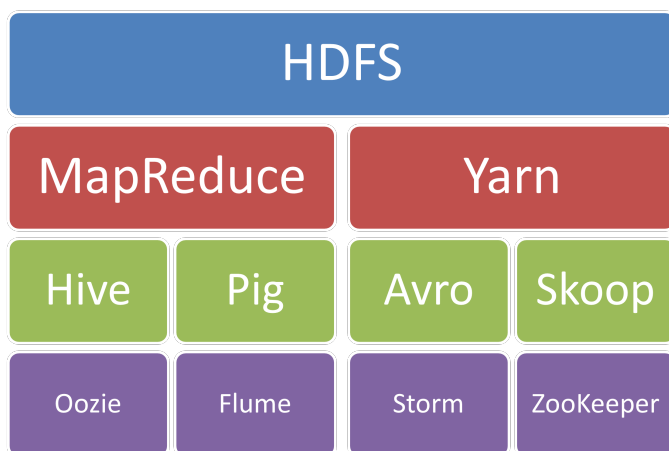


Figura 5.4: Componentes básicos de una Arquitectura Hadoop

5.7.1. Principales Componentes

HDFS

Es el componente principal del ecosistema Hadoop. Hace posible almacenar conjuntos de datos masivos con tipos de datos estructurados, semi-estructurados y no estructurados como imágenes, vídeo, datos de sensores, etc.

Es un sistema de archivos distribuido y está optimizado para obtener un alto rendimiento y trabajar con máxima eficiencia cuando se leen archivos grandes. Para obtener este rendimiento, utiliza tamaños de bloque inusualmente grandes y optimización de localización de los datos para reducir la E/S de red.

Con el fin de ofrecer una visión de los recursos como una sola unidad crea una capa de abstracción como un sistema de ficheros único. Está basado en la idea de que mover el procesamiento es mucho más rápido, fácil y eficiente que mover grandes cantidades de datos, que pueden producir altas latencias y congestión en la red.

La escalabilidad y la disponibilidad también son rasgos clave de HDFS, logrados en parte debido a la replicación de datos y la tolerancia a fallos. HDFS replica archivos en un número configurable, es tolerante a fallos tanto de software como de hardware y repite automáticamente los bloques de datos en los nodos que han fallado.

Para conseguir una alta escalabilidad, HDFS usa almacenamiento local que escala horizontalmente. Soporta miles de nodos y lo más típico en un clúster es desplegar decenas o cientos de nodos y manejar cientos de terabytes, con la capacidad de escalar a decenas de petabytes. Para mantener la integridad de los datos, HDFS almacena por defecto tres copias de cada bloque de datos

MapReduce

MapReduce es un framework distribuido basado en lotes modelado a partir del documento de Google sobre MapReduce. Le permite paralelizar el trabajo sobre una gran cantidad de datos sin procesar, como combinar registros web con datos relacionales de una base de datos OLTP a modelar cómo los usuarios interactúan con su sitio web. Este tipo de trabajo, que podría llevar días o más tiempo usando técnicas convencionales de programación en serie, se puede reducir a minutos usando MapReduce en un clúster Hadoop. El modelo MapReduce simplifica el procesamiento paralelo al abstraer las complejidades involucradas en el trabajo con sistemas distribuidos, como la paralelización computacional, la distribución del trabajo y el manejo de hardware y software poco confiables. Con esta abstracción, MapReduce permite al programador centrarse en abordar las necesidades del negocio en lugar de enredarse en complicaciones del sistema distribuido.

Hadoop MapReduce es un paradigma de procesamiento de datos caracterizado por dividirse en dos fases o pasos diferenciados: *Map* y *Reduce*. Estos subprocesos asociados a la tarea se ejecutan de manera distribuida, en diferentes nodos de procesamiento o esclavos. Para controlar y gestionar su ejecución, existe un proceso *Master* o *Job Tracker*. También es el encargado de aceptar los nuevos trabajos enviados al sistema por los clientes. Este sistema de procesamiento se apoya en tecnologías de almacenamiento de datos distribuidas, en cuyos nodos se ejecutan estas operaciones de tipo *map* y *reduce*. HDFS es el encargado de almacenar los ficheros divididos en bloques de datos. También proporciona la división previa de los datos en bloques que necesita *MapReduce* para ejecutar. Los resultados de procesamiento se pueden almacenar en el mismo sistema de almacenamiento o bien en una base de datos o sistema externo.

YARN

YARN (Yet Another Resource Negotiator) es una pieza fundamental en el ecosistema Hadoop. Es el framework que permite a Hadoop soportar varios motores de ejecución incluyendo MapReduce, y proporciona un planificador de los trabajos que se encuentran en ejecución en el clúster. Esta mejora de Hadoop también es conocida como Hadoop 2. Yarn separa las dos funcionalidades principales: la gestión de recursos y la planificación y monitorización de trabajos. Con esta idea, es posible tener un gestor global (Resource Manager) y un Application Master por cada aplicación.

La necesidad de YARN se basa en solventar las desventajas que tenía MapReduce. MapReduce es un arco distribuido y modelo de programación que permite realizar trabajos paralelos basados en lotes en un clúster de múltiples nodos. A pesar de ser muy eficiente en lo que hace, tiene algunas desventajas; principalmente porque está basado en lotes y, como resultado, no era adecuado para el procesamiento de datos en tiempo real o incluso en tiempo casi real.

YARN cambia todo esto al hacerse cargo de las partes de scheduling de MapReduce. Con esto solventa esa carencia de MapReduce. Principalmente, se encarga de responder a solicitudes de cliente para crear un contenedor y supervisar los contenedores que se están ejecutando y finalizarlos si es necesario para liberar recursos.

5.7.2. Ecosistema

Hadoop no es un proyecto Open Source independiente, es más bien un complejo ecosistema de proyectos muy diversos que trabajan a la par. Su objetivo es crear un conjunto común de servicios capaces de transformar lo que llamamos *commodity hardware* (hardware de bajo coste, sin capacidad de redundancia), en un servicio coherente que permita almacenar de forma redundante Petabytes de datos y procesarlos eficientemente.

- **Pig:** es un lenguaje de alto nivel que traduce a “MapReduce”. Convierte una descripción de alto nivel de cómo deben ser procesados los datos en Jobs de MapReduce, sin necesidad de tener que escribir largas cadenas de jobs cada vez, mejorando notablemente la productividad de los desarrolladores.
- **Hive:** Convierte una operación en lenguaje SQL en una operación Pig o directamente en una operación MapReduce.
- **HBase:** HBase ofrece un motor de procesamiento en memoria que le agiliza enormemente las operaciones de lectura escritura sobre Hadoop, permitiendo así trabajar con datos en streaming. También permite trabajar con bases de datos noSQL. Se puede acceder a HBase desde Hive, Pig y MapReduce y usa HDFS para almacenar la información, por tanto es completamente tolerante a fallos.
- **Zookeeper:** es otro proyecto de Apache que almacena y facilita servicios de coordinación para distintos servidores.

- **HCatalog**: Es un servidor de metadatos con algunas mejoras. Extrae los metadatos de Hive para que también se pueda acceder a ellos Pig y MapReduce. HCatalog puede acceder a los datos en el estándar HDFS o bien en HBase.
- **Ambari, Ganglia, Nagios** ofrecen una interfaz de acceso al clúster.
- **Sqoop**, que permite ejecutar aplicaciones MapReduce que introducen o extraen información de bases de datos SQL (por tanto, estructuradas).
- **Flume** sirve para introducir datos en streaming en Hadoop. Si tenemos servidores que generan datos de forma continua, se puede usar Flume para almacenarlos en HDFS. (pueden ser datos semiestructurados o no estructurados).
- **Oozie** es un gestor de workflow. Te permite definir cuándo quieres que tus jobs MapReduce se ejecuten, de forma programada o cuando haya disponibles nuevos datos.
- **Storm**. Apache Storm es un proyecto que permite organizar el procesamiento garantizado de varios eventos en tiempo real. Por ejemplo, Storm puede usarse para analizar flujos de datos en tiempo real, realizar tareas de aprendizaje automático, organizar cálculos continuos, implementar RPC, ETL, etc.

En cuanto a las distribuciones de Hadoop se detallan las más importantes:

- **Apache**: El soporte se limita a la buena voluntad de la comunidad y las respuestas a los problemas suelen ser rápidas. La llegada de Apache Ambari ha hecho que la administración sea más sencilla.
- **Cloudera**: Es la distribución de Hadoop más importante. Ofrece uno de los mejores soportes de las distribuciones aquí presentadas. Ha estado innovando en Hadoop mediante el desarrollo de proyectos allí donde Hadoop se ha mostrado más débil. Uno de estos ejemplos es Impala, que ofrece un sistema SQL en Hadoop similar a Hive pero que ofrece una experiencia de usuario casi en tiempo real. Existen otros muchos proyectos aportados por Cloudera.
- **HortonWorks**: También ofrece un soporte notable. En cuanto a la innovación ha adoptado un enfoque ligeramente diferente a Cloudera. En vez de crear proyectos nuevos opta por mejorar lo existente. También es el impulsor de YARN.
- **MapR**: Tiene menos desarrolladores que las otras distribuciones. Desde el principio decidió que HDFS no era un sistema de archivos idóneo para su empresa y desarrolló uno propietario. Su base principal es propietaria por lo que es más difícil para sus ingenieros explorar, corregir y contribuir con parches para la comunidad. Uno de sus aspectos destacados es estar disponible en la nube de Amazon como alternativa a Elastic MapReduce de Amazon e integrarse con Compute Cloud de Google.



5.8. Spark

Apache Spark es un framework open source basado en computación por clusters para el tratamiento de un gran número de datos. Desarrollado en la Universidad de California por AMPLAB y mantenido hoy en día por la fundación Apache tras la donación del código base por parte del laboratorio que lo diseñó e implementó [FSW16].

Spark ofrece un gran número de ventajas frente a otros sistemas:

- **Velocidad:** El procesamiento rápido de datos es una necesidad cuando hablamos de Big Data. Todo el tiempo que se utilice para el tratamiento de nuestros datos es dinero, en proyectos pequeños muchas veces este parámetro es irrelevante, pero no en Big Data dónde tenemos que procesar grandes cantidades de datos en tiempo real.
- **Facilidad de uso:** Spark puede usarse en multitud de plataformas. Aunque estuviera escrito originalmente para Scala, podemos desarrollar aplicaciones para Java, Python, SQL y R dando mucha versatilidad a los usuarios de cómo tratar sus datos. Poder combinar diferentes estructuras de datos como por ejemplo SQL y un DataFrame de Python Data Analysis Library (Pandas) es algo que da mucha libertad a la hora de recoger fuentes de datos en casos reales. Ya en proyectos de empresas medianas nos encontraremos con bases de datos de estructuras totalmente diferentes, por lo que podemos imaginarnos como de diferente es la información que se recoge en cualquier proyecto basado en Big Data.
- **Dinámico en naturaleza** Con los 80 operadores de alto nivel, el paralelismo de las aplicaciones desarrolladas en Spark es altísimo. Esto también es muy importante para estos sistemas ya que no vamos a usar todos los datos para un único modelo, si no que tendremos diferentes modelos que comparten las soluciones que dan entre ellos para así sacar los resultados finales.
- **Computación en memoria:** No tener que leer los datos constantemente y poder almacenarlos en memoria caché es una poderosa herramienta que hace que los cuellos de botella que anteriormente se podían generar debido a la falta de cómputo de la CPU puedan solucionarse, además de abaratar el coste del hardware.

Alguno de los inconvenientes que presenta Spark son los siguientes:

- **Gran uso de memoria RAM:** Ya como hemos comentado en las ventajas, Spark permite hacer computaciones en memoria caché, pero esto en si mismo también puede ser una desventaja por diversas razones. Una de ellas es que mucho del hardware que las empresas tienen para hacer tareas de aprendizaje automático no suele estar optimizado para hacer un uso tan grande de caché, si no que ponen más potencia en el procesador ya que el software alternativo a Spark requiere de más poder de procesamiento.

- No existen estructuras de datos propias de Spark como anteriormente se ha mencionado, lo que muchas veces origina problemas de logística sobre cuál usar, aparte de depender de librerías externas.
- Optimización manual: Relacionado con el uso de caché y elección de estructura de datos. Si queremos utilizar todo el potencial de Spark tendremos que de manera manual gestionar toda la parte de utilización de memoria, lo cual puede quitar mucho tiempo de desarrollo y lo más importante, puede dar lugar a errores difícilmente detectables por un mal aprovechamiento de los recursos.
- Problemas con ficheros externos Spark está pensado para pocos ficheros, pero de tamaño muy grande, no para muchos ficheros de tamaño pequeño, por lo que si nuestros datos están divididos en multitud de bases de datos más pequeñas, habría que hacer transformaciones previas a Spark para poder hacer un uso más correcto

Spark es muy adecuado para proyectos de Big Data compuestos de ficheros de tamaño muy elevado, en una arquitectura que disponga de una cantidad considerable de memoria RAM y caché y que se tenga que ajustar a numerosas estructuras de datos diferentes. Spark no es tan bueno en proyectos que no dispongan de personal dedicado a la optimización de recursos o en modelos pequeños a los cuáles podríamos aplicar otras técnicas de minería de datos más clásicas.

Bibliografía

- [BGS⁺11] Dhruba Borthakur, Jonathan Gray, Joydeep Sen Sarma, Kannan Muthukkaruppan, Nicolas Spiegelberg, Hairong Kuang, Karthik Ranganathan, Dmytro Molkov, Aravind Menon, Samuel Rash, et al. Apache hadoop goes realtime at facebook. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of data*, pages 1071–1080, 2011.
- [CDK05] George F Coulouris, Jean Dollimore, and Tim Kindberg. *Distributed systems: concepts and design*. Pearson Education, 2005.
- [DK14] Manjula Dyavanur and Kavita Kori. Fault tolerance techniques in big data tools: a survey. *International Journal of Innovative Research in Computer and Communication Engineering IJIRCCCE*, 2, 2014.
- [FSW16] Jian Fu, Junwei Sun, and Kaiyuan Wang. Spark—a big data processing platform for machine learning. In *2016 International Conference on Industrial Informatics-Computing Technology, Intelligent Technology, Industrial Information Integration (ICIICII)*, pages 48–51. IEEE, 2016.
- [HK10] Christina N Hoefler and Georgios Karagiannis. Taxonomy of cloud computing services. In *2010 IEEE Globecom Workshops*, pages 1345–1350. IEEE, 2010.
- [JPS12] Ravi Jhavar, Vincenzo Piuri, and Marco Santambrogio. A comprehensive conceptual system-level approach to fault tolerance in cloud computing. In *2012 IEEE International Systems Conference SysCon 2012*, pages 1–5. IEEE, 2012.

- [KSV⁺21] Awais Khan, Hyogi Sim, Sudharshan S Vazhkudai, Ali R Butt, and Youngjae Kim. An analysis of system balance and architectural trends based on top500 supercomputers. In *The International Conference on High Performance Computing in Asia-Pacific Region*, pages 11–22, 2021.
- [PF07] Radu Prodan and Thomas Fahringer. *Grid computing: experiment management, tool integration, and scientific workflows*, volume 4340. Springer, 2007.
- [Sin19] Manjeet Singh. An overview of grid computing. In *2019 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, pages 194–198. IEEE, 2019.
- [SKRC10] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The hadoop distributed file system. In *2010 IEEE 26th symposium on mass storage systems and technologies (MSST)*, pages 1–10. Ieee, 2010.
- [WDDL16] Xing Wu, Zhikang Du, Shuji Dai, and Yazhou Liu. The fault tolerance of big data systems. In *International Workshop on Management of Information, Processes and Cooperation*, pages 65–74. Springer, 2016.
- [Whi12] Tom White. *Hadoop: The definitive guide*. .°Reilly Media, Inc.", 2012.